

AFRILANG Stdlib

std/c/simrandomwalk

Bibliothèque AFRILANG `std/c/simrandomwalk` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : randSum, ra

```
`import "std/c/simrandomwalk" · `use simrandomwalk`
```

- `randSum(v list number) ? number`
- `randMean(v list number) ? number`
- `randMin(v list number) ? number`
- `randMax(v list number) ? number`
- `randVar(v list number) ? number`
- `randStd(v list number) ? number`
- `randNorm(v list number) ? list number`
- `simulate(steps number, seed number) ? list number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG ``std/c/simrandomwalk`` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : `randSum`, `ra`

```
`import "std/c/simrandomwalk" . `use simrandomwalk`  
- `randSum(v list number) ? number`  
- `randMean(v list number) ? number`  
- `randMin(v list number) ? number`  
- `randMax(v list number) ? number`  
- `randVar(v list number) ? number`  
- `randStd(v list number) ? number`  
- `randNorm(v list number) ? list number`  
- `simulate(steps number, seed number) ? list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/simrandomwalk"  
use simrandomwalk
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? randSum(v list number) ? number  
? randMean(v list number) ? number  
? randMin(v list number) ? number  
? randMax(v list number) ? number  
? randVar(v list number) ? number  
? randStd(v list number) ? number  
? randNorm(v list number) ? list  
? simulate(steps number, seed number) ? list
```

4. Exemples d'utilisation

```
import "std/c/simrandomwalk"  
use simrandomwalk  
  
create result = randSum(/* args */)   
say result  
  
create result = randMean(/* args */)   
say result  
  
create result = randMin(/* args */)   
say result  
  
create result = randMax(/* args */)   
say result  
  
create result = randVar(/* args */)   
say result  
  
create result = randStd(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez ``import "std/c/simrandomwalk"`` en tête de fichier.
- ? Lancez ``afirang check`` avant de livrer.
- ? Combinez avec les modules `stdlib` de la même catégorie.
- ? Voir `/stdlib/` pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module simrandomwalk
function randSum(v list number) returns number
end
function randMean(v list number) returns number
end
function randMin(v list number) returns number
end
function randMax(v list number) returns number
end
function randVar(v list number) returns number
end
function randStd(v list number) returns number
end
function randNorm(v list number) returns list number
end
function simulate(steps number, seed number) returns list number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/simrandomwalk` (tier complex, category complex). Exports 8 function(s): randSum, randMean, randM

```
`import "std/c/simrandomwalk" . `use simrandomwalk`  
- `randSum(v list number) ? number`  
- `randMean(v list number) ? number`  
- `randMin(v list number) ? number`  
- `randMax(v list number) ? number`  
- `randVar(v list number) ? number`  
- `randStd(v list number) ? number`  
- `randNorm(v list number) ? list number`  
- `simulate(steps number, seed number) ? list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/simrandomwalk"  
use simrandomwalk
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? randSum(v list number) ? number  
? randMean(v list number) ? number  
? randMin(v list number) ? number  
? randMax(v list number) ? number  
? randVar(v list number) ? number  
? randStd(v list number) ? number  
? randNorm(v list number) ? list  
? simulate(steps number, seed number) ? list
```

4. Usage examples

```
import "std/c/simrandomwalk"  
use simrandomwalk  
  
create result = randSum(/* args */)   
say result  
  
create result = randMean(/* args */)   
say result  
  
create result = randMin(/* args */)   
say result  
  
create result = randMax(/* args */)   
say result  
  
create result = randVar(/* args */)   
say result  
  
create result = randStd(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/simrandomwalk"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module simrandomwalk
function randSum(v list number) returns number
end
function randMean(v list number) returns number
end
function randMin(v list number) returns number
end
function randMax(v list number) returns number
end
function randVar(v list number) returns number
end
function randStd(v list number) returns number
end
function randNorm(v list number) returns list number
end
function simulate(steps number, seed number) returns list number
end
end
```